

EcoCraftingTool

Audit date - 2026-05-08 18:25

Angular ^21.2.0 - 83 TS files - 95,004 LOC

0/100

Grade F

Critique - le projet necessite une intervention majeure.

53 issues detected

8

CRITICAL

32

IMPORTANT

13

MINOR

Project overview

Angular version	^21.2.0
TypeScript files	83
HTML templates	14
Components / Services / Modules	26 / 16 / 0
Pipes / Guards	2 / 0
Total lines of code	95,004
Tests detected	No

Severity summary



Critical: 8

Important: 32

Minor: 13

Issues detail

Memory Leaks

CRITICAL

MEM001 Subscription sans unsubscribe (8)

Une subscription sans unsubscribe/takeUntil crée un memory leak.

Fix: Utiliser `takeUntil(this.destroy$)` ou `takeUntilDestroyed()` (Angular 16+) ou le `async`` pipe dans le template.

OCCURRENCES

[src/app/service/price-calculator-server.service.spec.ts:44](#)

```
service.getAllRecipes(true).subscribe(result => {
```

[src/app/service/price-calculator-server.service.spec.ts:70](#)

```
service.getAllItems(true).subscribe(result => {
```

[src/app/service/price-calculator-server.service.spec.ts:88](#)

```
service.attemptConnection(serverConfig).subscribe(result => connectionSucceeded = result);
```

[src/app/service/price-calculator-server.service.spec.ts:106](#)

```
service.attemptConnection(serverConfig).subscribe(result => connectionSucceeded = result);
```

[src/app/service/price-calculator-server.service.spec.ts:128](#)

```
service.attemptConnection(serverConfig).subscribe(result => connectionSucceeded = result);
```

[src/app/header/header.component.ts:63](#)

```
this.releaseNotesService.getReleases().subscribe(releases => {
```

[src/app/header/header.component.ts:112](#)

```
dialogRef.afterClosed().subscribe((result: ServerDialogResult) => {
```

[src/app/header/server/server-dialog.component.ts:121](#)

```
.subscribe({
```

IMPORTANT

JS001 setTimeout/setInterval sans cleanup (5)

Un `setTimeout`` ou surtout `setInterval`` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction du composant. Sur une SPA Angular, accumuler des intervalles oublies = memory leak progressif + appels reseau fantomes. Different de RxJS subscriptions (gere par MEM001).

Fix: Garder la reference (`this.timerId = setTimeout(...)`) et appeler `clearTimeout(this.timerId)` dans `ngOnDestroy()`. Ou mieux : utiliser `interval(N).pipe(takeUntilDestroyed())` (Angular 16+) qui s'auto-nettoie.

OCCURRENCES

[src/app/service/locale.service.ts:79](#)

```
await new Promise(resolve => setTimeout(resolve));
```

[src/app/service/locale.service.ts:88](#)

```
await new Promise(resolve => setTimeout(resolve));
```

[src/app/service/locale.service.ts:97](#)

```
await new Promise(resolve => setTimeout(resolve));
```

[src/app/crafting/skills/skills.component.ts:79](#)

```
setTimeout(() => {
```

[src/app/crafting/skills/skills.component.ts:114](#)

```
setTimeout(() => {
```

Type Safety

IMPORTANT

TYPE001 Usage de 'any' TypeScript (9)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

Fix: Typier explicitement (interface, type, générique). Utiliser `unknown` si le type est vraiment inconnu.

OCCURRENCES

[src/app/test-utils.ts:13](#)

```
export function provideMockDialogData(data: any = null): Provider {
```

[src/app/test-utils.ts:20](#)

```
export function provideDialogTestingDependencies(dialogData?: any): Provider[] {
```

[src/app/service/storage.service.ts:29](#)

```
get(key: string): any;
```

[src/app/service/locale.service.ts:30](#)

```
get(key: string): any;
```

[src/app/service/image.service.ts:71](#)

```
getImgStyle(nameID: string, imageFile: string, xPos: number, yPos: number, size: number, filter...
```

[src/app/service/image.service.ts:78](#)

```
getProfitImgStyle(): any {
```

[src/app/service/image.service.ts:84](#)

```
getCalorieImgStyle(): any {
```

[src/app/service/image.service.ts:90](#)

```
getTableImgStyle(table: CraftingTable): any {
```

... and 1 more occurrence(s).

IMPORTANT

TYPE002 Cast 'as any' explicite (1)

Un cast `as any` désactive volontairement TypeScript pour cette expression. Différent de `: any` (TYPE001) qui déclare un type ambigu : `as any` est un acte explicite de bypass du vérificateur. Apparaît souvent quand un dev se bat avec un type de librairie tiers, ou quand un payload API renvoie une structure non typée. Le problème : le compilateur ne peut plus garantir que les accès suivants (`.foo`, `.bar()`) sont valides - un refactor de la source ne mettra plus à jour les usages, et un null/undefined dans le payload ne sera pas détecté au build.

Fix: 1) Si la forme est connue : déclarer une `interface` ou `type` et caster vers ce type (`as User`). 2) Si la forme est partiellement connue : `as Partial<User>` ou `as Pick<User, 'id'|'name'>`. 3) Si la forme est vraiment inconnue : caster vers `unknown` puis valider avec un type guard avant l'accès (`if (typeof x === 'object' && x !== null && 'id' in x)`). 4) Pour les réponses HTTP : utiliser `http.get<MyType>(url)` ou un schéma runtime (zod, yup) pour valider la forme avant cast. Le cast `as unknown` est préférable à `as any` car il force au moins une étape de validation explicite.

OCCURRENCES

[src/app/service/locale.service.ts:120](#)

```
const wn = window.navigator as any;
```

Architecture

IMPORTANT**ARCH001 HttpClient dans un composant (11)**

Les appels HTTP dans les composants mélangent les responsabilités. Difficile à tester et réutiliser.

Fix: Déplacer les appels HTTP dans des services dédiés. Les composants ne consomment que des observables.

OCCURRENCES

[src/app/app.config.ts:2](#)

```
import {provideHttpClient} from '@angular/common/http';
```

[src/app/app.config.ts:8](#)

```
provideHttpClient(),
```

[src/app/service/price-calculator-server.service.spec.ts:5](#)

```
import {HttpTestingController, provideHttpClientTesting} from '@angular/common/http/testing';
```

[src/app/service/price-calculator-server.service.spec.ts:6](#)

```
import {provideHttpClient} from '@angular/common/http';
```

[src/app/service/price-calculator-server.service.spec.ts:20](#)

```
providers: [provideHttpClient(), provideHttpClientTesting()]
```

[src/app/service/release-notes.service.spec.ts:4](#)

```
import {provideHttpClient} from '@angular/common/http';
```

[src/app/service/release-notes.service.spec.ts:12](#)

```
providers: [provideHttpClient(), provideDialogTestingDependencies()]
```

[src/app/header/header.component.spec.ts:4](#)

```
import {provideHttpClient} from '@angular/common/http';
```

... and 3 more occurrence(s).

Accessibilité

IMPORTANT**A11Y002 Click sur element non-interactif (6)**

Un (click) sur un <div> ou sans role ni tabindex est inaccessible au clavier et aux lecteurs d'écran. L'utilisateur ne peut pas activer l'action sans souris.

Fix: Soit utiliser un vrai <button> (ou <a> si c'est une navigation), soit ajouter `role="button" tabindex="0"` + handler `(keydown.enter)` et `(keydown.space)`.

OCCURRENCES

[src/app/header/export/export-dialog.component.html:4](#)

```
<span class="ml-auto material-icons cursor-pointer hover:opacity-80" (click)="close()">close</s...
```

[src/app/header/release-notes/release-notes-dialog.component.html:3](#)

```
<span class="ml-auto material-icons cursor-pointer hover:opacity-80" (click)="close()">close</s...
```

[src/app/header/settings/settings-dialog.component.html:3](#)

```
<span class="ml-auto material-icons cursor-pointer hover:opacity-80" (click)="close()">close</s...
```

[src/app/header/import/import-dialog.component.html:4](#)

```
<span class="ml-auto material-icons cursor-pointer hover:opacity-80" (click)="close()">close</s...
```

[src/app/header/server/server-dialog.component.html:12](#)

```
<span class="ml-auto material-icons cursor-pointer hover:opacity-80" (click)="close()">close</s...
```

[src/app/crafting/recipe-dialog/recipe-dialog.component.html:3](#)

```
<span class="material-icons cursor-pointer hover:opacity-80" (click)="close()">close</span>
```

Code Quality

MINOR**DEBUG001 console.log en production (13)**

Les console.log oubliés exposent des données internes en prod et polluent la console.

Fix: Supprimer ou remplacer par un service de logging. Configurer `build.optimization.scripts` pour stripper en prod.

OCCURRENCES

[src/main.ts:6](#)

```
.catch((err) => console.error(err));
```

[src/assets/data/util/data-utils.ts:11](#)

```
console.error(`Crafting table ${nameID} not found`);
```

[src/assets/data/util/data-utils.ts:19](#)

```
console.error(`Skill ${nameID} not found`);
```

[src/assets/data/util/data-utils.ts:27](#)

```
console.error(`Item ${nameID} not found`);
```

[src/app/service/crafting.service.ts:289](#)

```
console.warn(`Could not compute prices for all recipes after ${maxLoops} loops`);
```

[src/app/service/locale.service.ts:156](#)

```
console.warn(`Could not find locale data for ${type}`);
```

[src/app/service/locale.service.ts:161](#)

```
console.debug(`Could not find locale entry for ${nameID} in ${type}`);
```

[src/app/service/price-calculator-server.service.ts:151](#)

```
console.debug(`New item found: ${id}`);
```

... and 5 more occurrence(s).

Refactoring plan

This week - Critical

- Subscription sans unsubscribe (MEM001)

Une subscription sans unsubscribe/takeUntil crée un memory leak.

This month - Important

- Usage de 'any' TypeScript (TYPE001)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

- HttpClient dans un composant (ARCH001)

Les appels HTTP dans les composants mélangent les responsabilités. Difficile à tester et réutiliser.

- Click sur element non-interactif (A11Y002)

Un (click) sur un <div> ou sans role ni tabindex est inaccessible au clavier et aux lecteurs d'écran. L'utilisateur ne p...

- setTimeout/setInterval sans cleanup (JS001)

Un `setTimeout` ou surtout `setInterval` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction...

- Cast 'as any' explicite (TYPE002)

Un cast `as any` desactive volontairement TypeScript pour cette expression. Different de `: any` (TYPE001) qui declare un type ...

On the roadmap - Minor

- console.log en production (DEBUG001)

Les console.log oubliés exposent des données internes en prod et polluent la console.

This report is produced by automated static analysis. It does not replace a thorough manual review by an experienced Angular developer.